

Informe: Fundamentos de la Programación Orientada a Objetos y Modelado

1. Introducción a las Clases

En la Programación Orientada a Objetos (POO), una **clase** es una plantilla, molde o plano a partir del cual se crean objetos. Define las características (atributos) y los comportamientos (métodos) que tendrán los objetos creados a partir de ella. Se puede pensar en una clase como un concepto abstracto, mientras que un objeto es la manifestación física y concreta de ese concepto (instancia).

2. Tipos de Clases

Dependiendo del lenguaje de programación y la arquitectura del software, existen varios tipos de clases con propósitos específicos:

- **Clases Concretas:** Son las clases estándar y más comunes. Se pueden instanciar directamente para crear objetos. Tienen todos sus métodos implementados.
- **Clases Abstractas:** Son clases que no pueden ser instanciadas directamente. Sirven como clases base (superclases) para que otras clases hereden de ellas. Suelen contener "métodos abstractos" que obligan a las clases hijas a implementar ese comportamiento.
- **Clases Estáticas:** Son clases que no necesitan ser instanciadas para acceder a sus miembros. Solo contienen métodos y atributos estáticos. Son útiles para agrupar funciones de utilidad (ej. una clase Math).
- **Clases Finales o Selladas (Sealed/Final):** Son clases de las cuales no se puede heredar. Evitan que otros programadores extiendan su funcionalidad, protegiendo la lógica interna.
- **Clases de Datos (Data Classes / Records):** Son clases diseñadas principalmente para almacenar datos, generando automáticamente métodos básicos como los de comparación, copia o representación en texto.

TEMA: Clases, tipos, atributos y métodos.

3. Atributos de una Clase

Los **atributos** (también llamados propiedades, campos o variables de instancia) representan el **estado** o las características de una clase. Son los datos que definen a cada objeto individual.

- **Ejemplo conceptual:** Si la clase es Coche, los atributos podrían ser color, marca, modelo y velocidadActual.
- **Visibilidad (Encapsulamiento):** Los atributos suelen protegerse mediante modificadores de acceso:
- *Públicos (Public):* Accesibles desde cualquier parte del programa.
- *Privados (Private):* Solo accesibles desde dentro de la misma clase. (Práctica recomendada).
- *Protegidos (Protected):* Accesibles dentro de la misma clase y por sus clases hijas (herencia).

4. Métodos de la Clase

Los **métodos** representan el **comportamiento** o las acciones que un objeto puede realizar. Son funciones definidas dentro de la clase que a menudo manipulan o utilizan los atributos del objeto.

- **Tipos de métodos comunes:**
 - **Constructor:** Un método especial que se ejecuta automáticamente cuando se crea un objeto. Se usa para inicializar los atributos.
 - **Getters (Accesores):** Métodos que devuelven el valor de un atributo privado.
 - **Setters (Mutadores):** Métodos que permiten modificar el valor de un atributo privado de forma controlada.
 - **Métodos operacionales:** Definen las acciones específicas del objeto (ej. acelerar(), frenar()).
 -

TEMA: Clases, tipos, atributos y métodos.

5. Modelado Práctico: Sistema Educativo

A continuación, se presenta el diseño de tres clases interrelacionadas para un sistema de gestión escolar, definiendo sus atributos y métodos esenciales.

5.1. Clase Alumno

Representa a los estudiantes matriculados en la institución.

- **Atributos:**
 - numeroMatricula (Cadena/Texto): Identificador único del estudiante.
 - nombreCompleto (Cadena/Texto): Nombre del alumno.
 - fechaNacimiento (Fecha): Para calcular la edad.
 - calificaciones (Lista/Arreglo): Historial de notas obtenidas.
- **Métodos:**
 - inscribirMateria(Materia m): Añade una nueva materia al horario del alumno.
 - entregarTarea(String nombreTarea): Simula la entrega de un trabajo.
 - calcularPromedio(): Devuelve el promedio actual basado en sus calificaciones.

5.2. Clase Docente

Representa a los profesores que imparten clases en la institución.

- **Atributos:**
 - númeroEmpleado (Cadena/Texto): Identificador único como trabajador.
 - nombreCompleto (Cadena/Texto): Nombre del profesor.
 - especialidad (Cadena/Texto): Área de conocimiento (ej. "Matemáticas", "Programación").
 - salario (Decimal): Sueldo asignado al docente.

TEMA: Clases, tipos, atributos y métodos.

- **Métodos:**
 - `impartirClase(Materia m)`: Acción de dictar la lección de una materia asignada.
 - `calificarAlumno(Alumno a, double nota)`: Asigna una nota al expediente de un estudiante.
 - `solicitarMaterial(String descripcion)`: Pide recursos para su labor académica.

5.3. Clase Materia

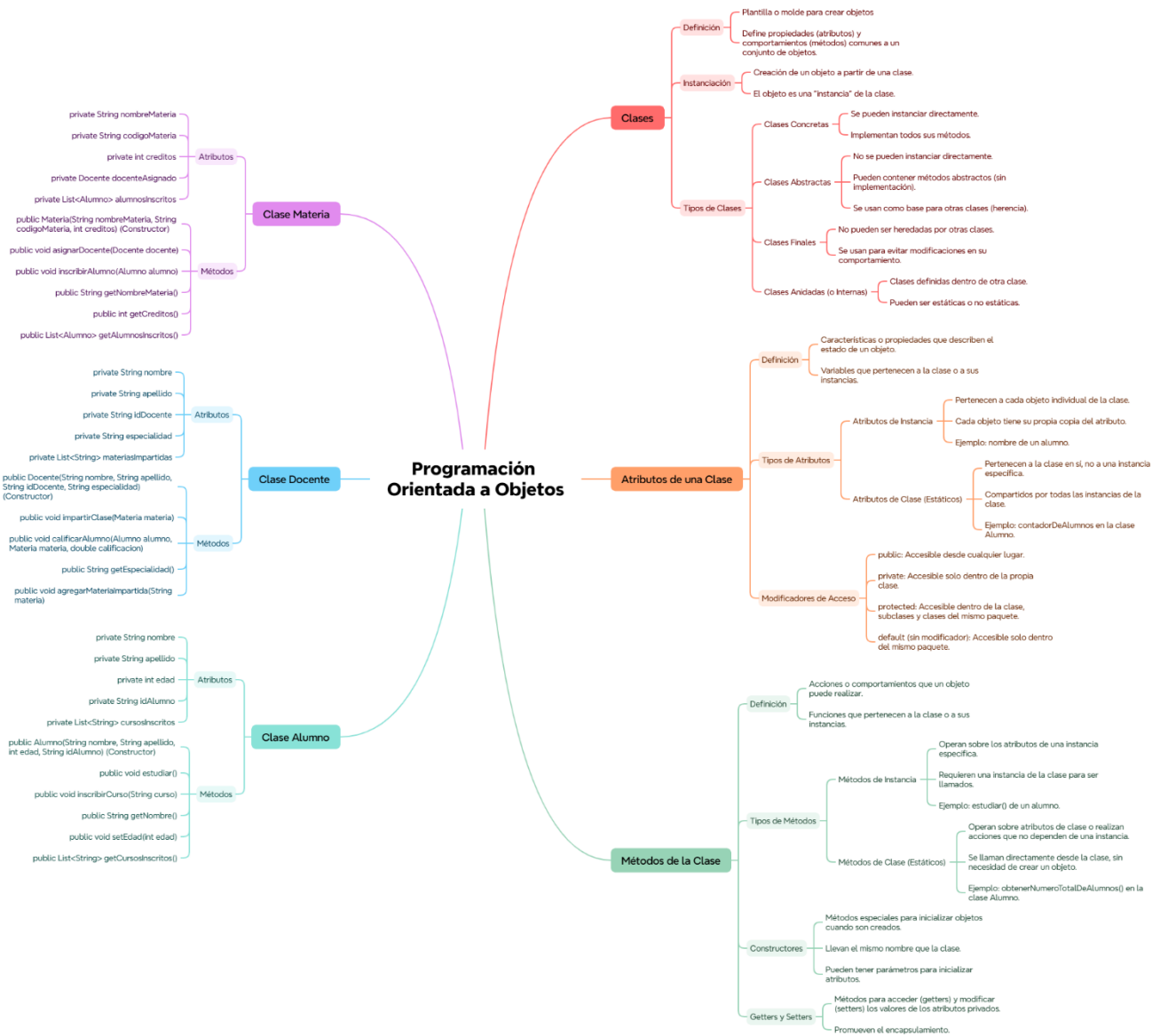
Representa las asignaturas o cursos disponibles en el plan de estudios.

- **Atributos:**
 - `codigoMateria (Cadena/Texto)`: Código único (ej. "MAT-101").
 - `nombre (Cadena/Texto)`: Nombre descriptivo (ej. "Álgebra Lineal").
 - `creditos (Entero)`: Valor académico de la materia.
 - `docenteTitular (Objeto Docente)`: El profesor asignado a impartirla.
 - `alumnosInscritos (Lista de Objetos Alumno)`: Estudiantes cursando la materia.
- **Métodos:**
 - `asignarDocente(Docente d)`: Define qué profesor impartirá el curso.
 - `agregarAlumno(Alumno a)`: Registra a un estudiante en la lista de la clase.
 - `generarListaAsistencia()`: Devuelve un reporte con los alumnos inscritos para el pase de lista.

Conclusión

La correcta definición de clases, atributos y métodos es el pilar para construir software robusto. En el ejemplo del sistema educativo, la separación de responsabilidades entre Alumno, Docente y Materia permite que el código sea modular, fácil de mantener y de expandir (por ejemplo, agregando después una clase Aula o Horario).

TEMA: Clases, tipos, atributos y métodos.



Presented with xmind